

Batch: HDA-5 Roll No.: 16010122158

Experiment No. 4

TITLE : Performing Graph Analytics

AIM: To analyze the structural properties of a real-world social network by constructing a graph representation, identifying key players and influential individuals through centrality measures, and detecting communities within the network using appropriate algorithms.

Expected OUTCOME of Experiment:

CO3: Perform the social data analytics

Books/ Journals/ Websites referred:

Students have to list.

Pre Lab/ Prior Concepts:

Students should have a basic understanding of:

Graph theory: Nodes, edges, directed and undirected graphs, weighted graphs.

Data structures: Lists, dictionaries.

Python programming: Basic syntax, data manipulation, libraries like NetworkX.

Statistical concepts: Mean, standard deviation, correlation.

Visualization techniques: Basic plotting using libraries like Matplotlib.

Procedure:

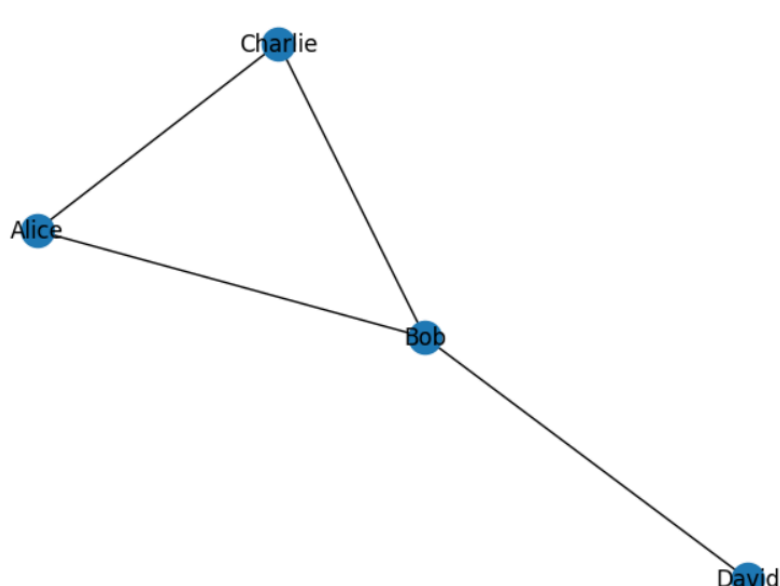
Building a Social Network Graph with NetworkX

```
1 import networkx as nx
2
3 # Create an empty graph
4 G = nx.Graph()
5
6 # Add nodes (individuals)
7 G.add_nodes_from(['Alice', 'Bob', 'Charlie', 'David'])
8
9 # Add edges (relationships)
10 G.add_edge('Alice', 'Bob')
11 G.add_edge('Alice', 'Charlie')
12 G.add_edge('Bob', 'Charlie')
13 G.add_edge('Bob', 'David')
14
15 # Print the graph
16 print(G.nodes())
17 print(G.edges())
```

```
[ 'Alice', 'Bob', 'Charlie', 'David' ]
[( 'Alice', 'Bob'), ( 'Alice', 'Charlie'), ( 'Bob', 'Charlie'), ( 'Bob', 'David')]
```

Visualizing the Graph

```
1 import matplotlib.pyplot as plt
2
3 # Draw the graph
4 nx.draw(G, with_labels=True)
5 plt.show()
```

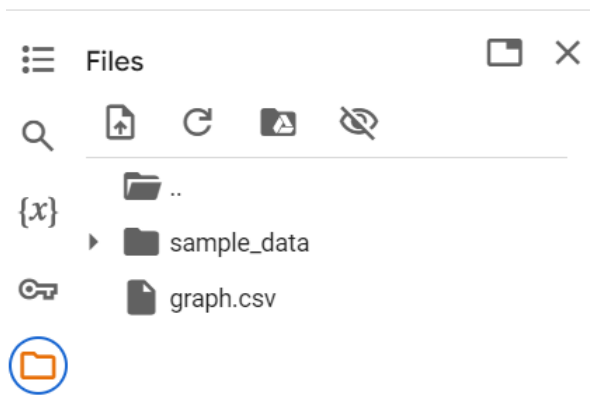


Exporting a NetworkX Graph to CSV

```

1  import networkx as nx
2  import pandas as pd
3
4  # Create a sample graph
5  G = nx.Graph()
6  G.add_edges_from([('A', 'B'), ('A', 'C'), ('B', 'C'), ('B', 'D')])
7
8  # Convert to edge list
9  edgelist = nx.to_edgelist(G)
10
11 # Create a pandas DataFrame, including a column for edge attributes
12 df = pd.DataFrame(edgelist, columns=['source', 'target', 'attributes'])
13
14 # Export to CSV
15 df.to_csv('graph.csv', index=False)
  
```

The csv file gets created



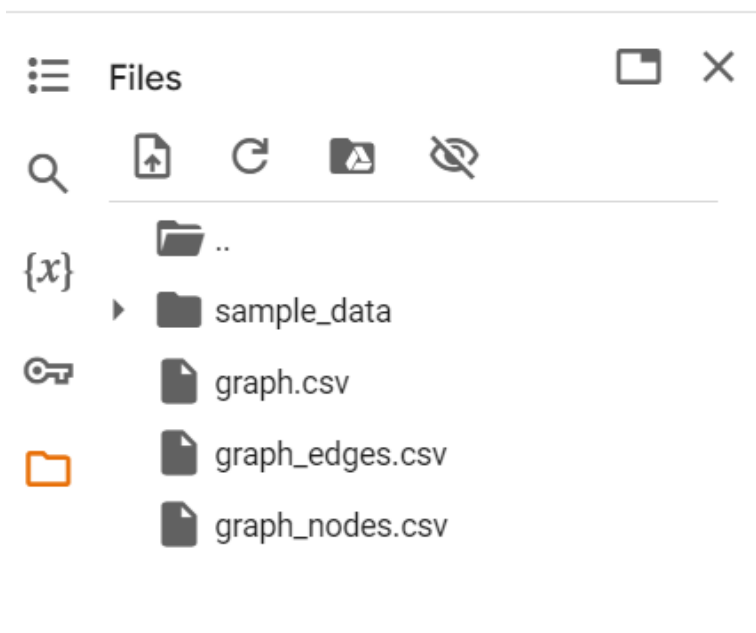
Contents of the csv file

	A	B	C	D
1	source	target	attributes	
2	A	B	{}	
3	A	C	{}	
4	B	C	{}	
5	B	D	{}	
6				
7				

Creating and exporting a NetworkX Graph with edge attributes and node attributes to a csv file

```
1 import networkx as nx
2 import pandas as pd
3
4 # Create a graph with node and edge attributes
5 G = nx.Graph()
6 G.add_edge('A', 'B', weight=2.5)
7 G.add_edge('A', 'C', weight=1.0)
8 G.nodes['A']['color'] = 'red'
9
10 # Convert to edge list with attributes
11 edgelist = [(u, v, d) for u, v, d in G.edges(data=True)]
12
13 # Create a pandas DataFrame
14 df = pd.DataFrame(edgelist, columns=['source', 'target', 'weight'])
15
16 # Add node attributes as a separate DataFrame if needed
17 node_attributes = pd.DataFrame.from_dict(dict(G.nodes(data=True)), orient='index')
18 node_attributes.columns = ['color']
19
20 # Export to CSV
21 df.to_csv('graph_edges.csv', index=False)
22 node_attributes.to_csv('graph_nodes.csv')
```

csv files get created



Contents of graph_edges.csv

	A	B	C	D
1	source	target	weight	
2	A	B	{'weight': 2.5}	
3	A	C	{'weight': 1.0}	
4				
5				

Contents of graph_nodes.csv

	A	B	
1		color	
2	A	red	
3			

Importing a graph from a csv file

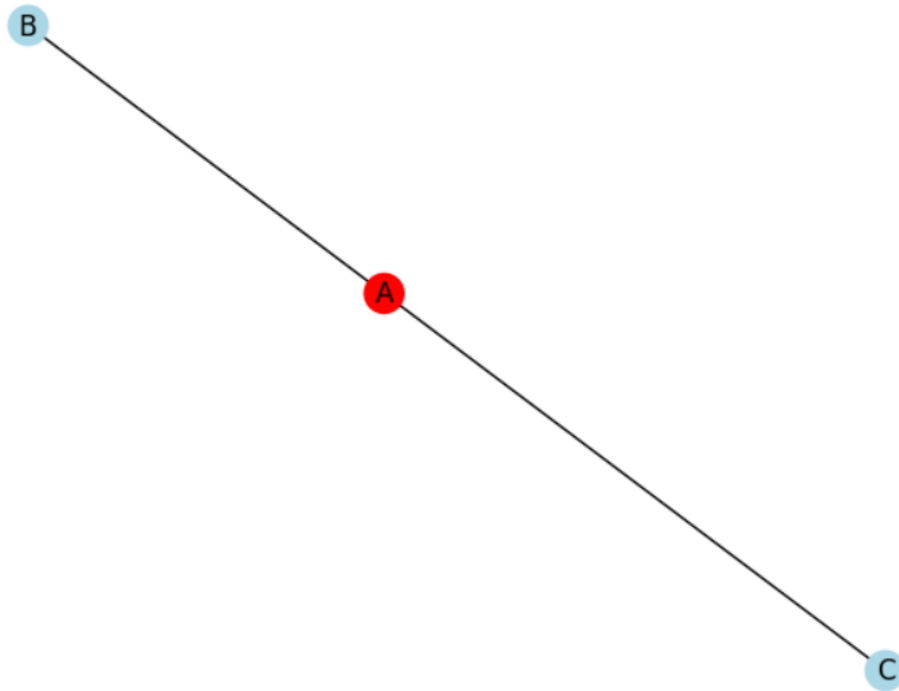
```

1 import pandas as pd
2 import networkx as nx
3 import matplotlib.pyplot as plt
4
5 # Read edge list from CSV
6 df_edges = pd.read_csv('graph_edges.csv')
7
8 df_nodes = pd.read_csv('graph_nodes.csv', index_col=0)
9
10 # Extract numeric weights from the 'weight' column (assuming they are stored as dictionaries)
11 # If your 'weight' column is just a number, remove this line
12 df_edges['weight'] = df_edges['weight'].apply(lambda x: float(x.strip("{}").split(": ")[1]) if isinstance(x, str) else x)
13
14 # Create a graph from the edge list
15 G = nx.from_pandas_edgelist(df_edges, source='source', target='target', edge_attr='weight')
16
17 # Add node attributes if available
18 if df_nodes is not None:
19     nx.set_node_attributes(G, df_nodes.to_dict('index'))
20
21 # Print the graph
22 print(G.nodes(data=True))
23 print(G.edges(data=True))
24
25 # Draw the graph
26 nx.draw(G, with_labels=True, node_color=[n[1]['color'] if 'color' in n[1] else 'lightblue' for n in G.nodes(data=True)])
27 plt.show()

```

Output (List of nodes and edges, and visualizing the imported graph)

```
[('A', {'color': 'red'}), ('B', {}), ('C', {})]  
[(('A', 'B', {'weight': 2.5}), ('A', 'C', {'weight': 1.0}))]
```



Graph Analytics

1. Degree centrality : The degree centrality for a node v is the fraction of nodes it is connected to. The degree centrality values are normalized by dividing by the maximum possible degree in a simple graph $n-1$ where n is the number of nodes in G .
2. Betweenness centrality : Betweenness centrality of a node v is the sum of the fraction of all-pairs shortest paths that pass through v . The betweenness centrality is normalized by dividing by the total number of shortest paths.
3. Edge betweenness centrality : Betweenness centrality of a node e is the sum of the fraction of all-pairs shortest paths that pass through e . The betweenness centrality is normalized by dividing by the maximum possible number of edges in a graph G .
4. Communities can be identified using the Girvan Newman algorithm, by successively deleting the edges with the highest betweenness centrality values.

Importing a graph from csv file and performing graph analytics

The graph in csv file:

	A	B	C
1	node1	node2	attribute
2	A	B	{}
3	A	C	{}
4	B	C	{}
5	C	D	{}
6	D	E	{}
7	D	F	{}
8	E	F	{}

Importing the graph, printing its edge list and visualizing it:

```
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt

# Read edge list from CSV
df_edges = pd.read_csv('new_graph_edges.csv')

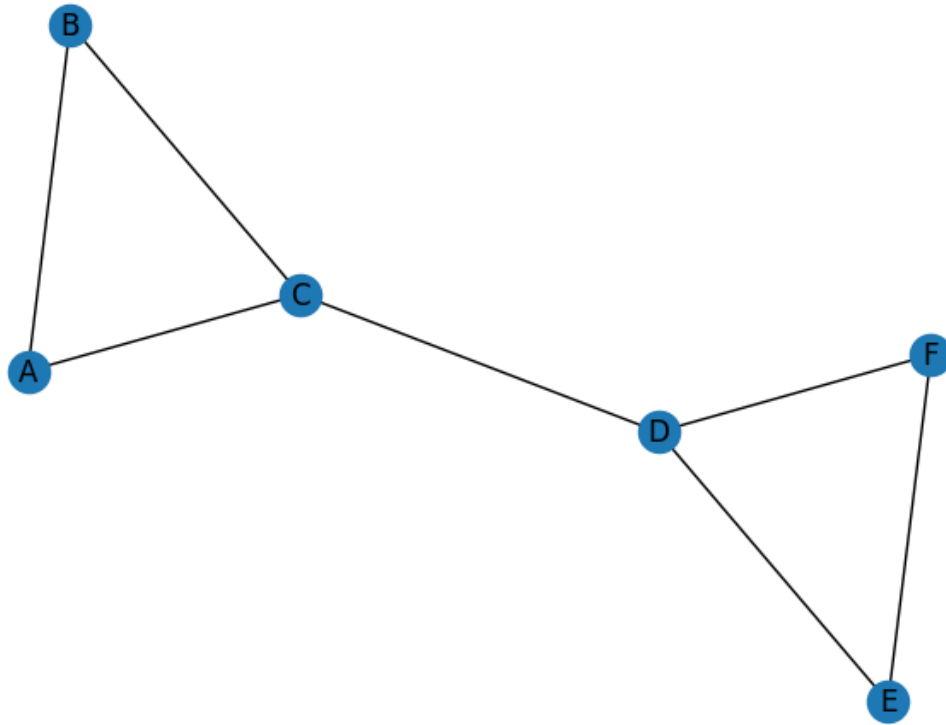
# Create a graph from the edge list
G = nx.from_pandas_edgelist(df_edges, source='node1', target='node2')

# Print the graph
print(G.nodes(data=True))
print(G.edges(data=True))

# Draw the graph
nx.draw(G, with_labels=True)
plt.show()
```

Output (graph details and visualization):

```
[('A', {}), ('B', {}), ('C', {}), ('D', {}), ('E', {}), ('F', {})]
[('A', 'B', {}), ('A', 'C', {}), ('B', 'C', {}), ('C', 'D', {}), ('D', 'E', {}), ('D', 'F', {}), ('E', 'F', {})]
```



Performing analytics on this graph:

```
# Basic graph properties
print("Number of nodes:", G.number_of_nodes())
print("Number of edges:", G.number_of_edges())

# Degree centrality
degrees = dict(G.degree())
print("\nDegree Centrality:", degrees)

# Betweenness centrality
betweenness = nx.betweenness centrality(G, normalized=False)
print("\nBetweenness Centrality:", betweenness)
betweenness = nx.betweenness centrality(G)
print("Normalized Betweenness Centrality:", betweenness)

# Closeness centrality
e_betweenness = nx.edge_betweenness centrality(G, normalized=False)
print("\nEdge Betweenness Centrality:", e_betweenness)
e_betweenness = nx.edge_betweenness centrality(G)
print("Normalized Edge Betweenness Centrality:", e_betweenness)
```

```
# Community detection (Girvan-Newman)
communities = nx.algorithms.community.girvan_newman(G)
try:
    top_level_communities = next(communities)
    print("\nCommunities after 1 step:", top_level_communities)

    top_level_communities = next(communities)
    print("\nCommunities after 2 steps:", top_level_communities)

    top_level_communities = next(communities)
    print("\nCommunities after 3 steps:", top_level_communities)

    top_level_communities = next(communities)
    print("\nCommunities after 4 steps:", top_level_communities)

    top_level_communities = next(communities)
    print("\nCommunities after 5 steps:", top_level_communities)

except StopIteration:
    print("\nNo more splits are possible.")
```

Output:

Number of nodes: 6

Number of edges: 7

Degree Centrality: {'A': 2, 'B': 2, 'C': 3, 'D': 3, 'E': 2, 'F': 2}

Betweenness Centrality: {'A': 0.0, 'B': 0.0, 'C': 6.0, 'D': 6.0, 'E': 0.0, 'F': 0.0}

Normalized Betweenness Centrality: {'A': 0.0, 'B': 0.0, 'C': 0.6000000000000001, 'D': 0.6000000000000001, 'E': 0.0, 'F': 0.0}

Edge Betweenness Centrality: {('A', 'B'): 1.0, ('A', 'C'): 4.0, ('B', 'C'): 4.0, ('C', 'D'): 9.0, ('D', 'E'): 4.0, ('D', 'F'): 4.0, ('E', 'F'): 1.0}

Normalized Edge Betweenness Centrality: {('A', 'B'): 0.06666666666666667, ('A', 'C'): 0.26666666666666666, ('B', 'C'): 0.26666666666666666, ('C', 'D'): 0.6, ('D', 'E'): 0.26666666666666666, ('D', 'F'): 0.26666666666666666, ('E', 'F'): 0.06666666666666667}

Communities after 1 step: ({'A', 'C', 'B'}, {'E', 'F', 'D'})

Communities after 2 steps: ({'A'}, {'C', 'B'}, {'E', 'F', 'D'})

Communities after 3 steps: ({'A'}, {'B'}, {'C'}, {'E', 'F', 'D'})

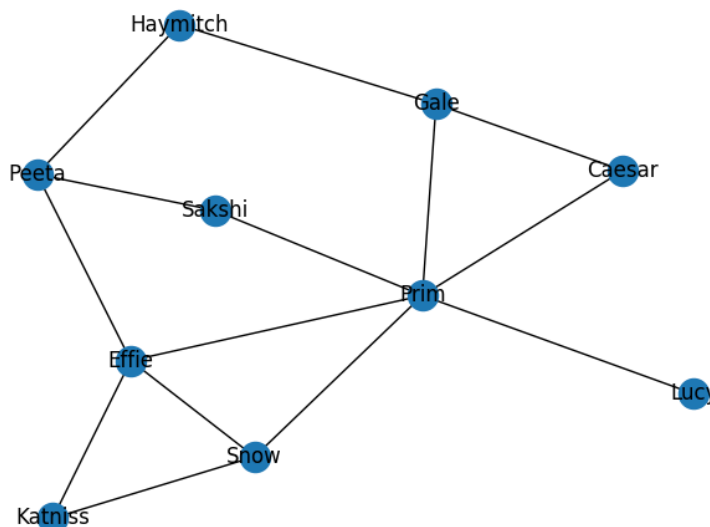
Communities after 4 steps: ({'A'}, {'B'}, {'C'}, {'D'}, {'E', 'F'})

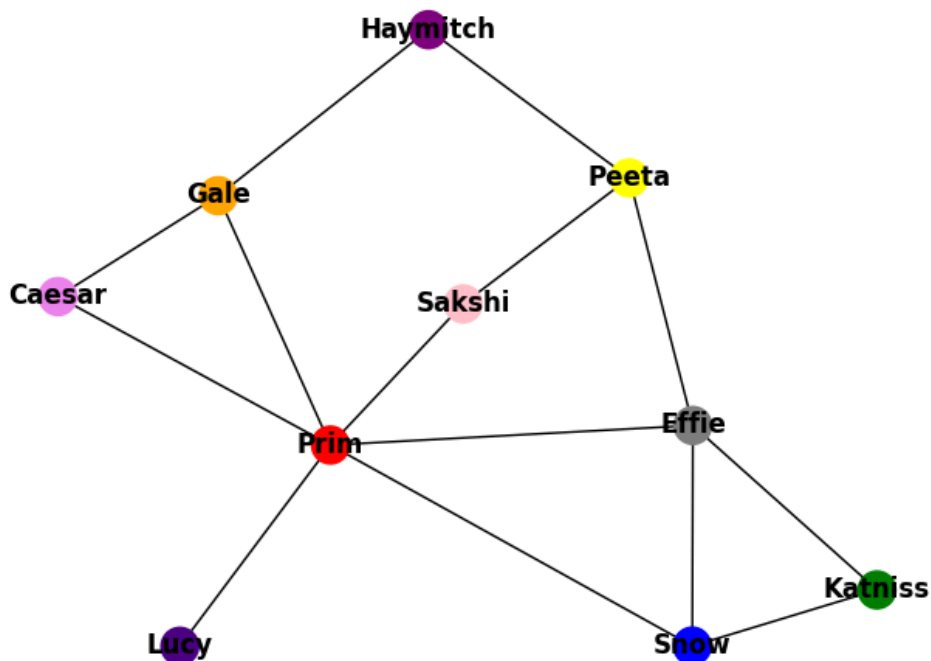
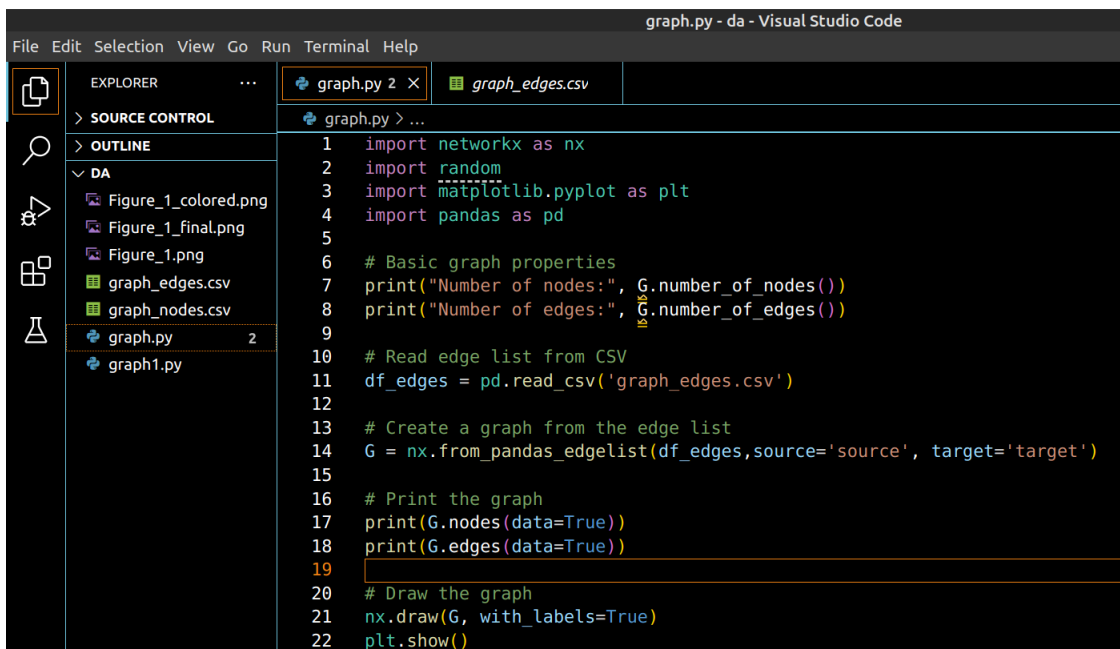
Communities after 5 steps: ({'A'}, {'B'}, {'C'}, {'D'}, {'E'}, {'F'})

Students have to perform all the tasks illustrated above by creating a social network graph with nodes labelled with their own names and their friends' names. The graph should have at least 10 nodes.

Students have to paste their code and screenshots of output and csv file below.

Implementation details:



```

graph.py - da - Visual Studio Code
File Edit Selection View Go Run Terminal Help
graph.py 2 x graph_edges.csv
EXPLORER
> SOURCE CONTROL
> OUTLINE
  DA
  Figure_1_colored.png
  Figure_1_final.png
  Figure_1.png
  graph_edges.csv
  graph_nodes.csv
  graph.py 2
  graph1.py
graph.py > ...
1 import networkx as nx
2 import random
3 import matplotlib.pyplot as plt
4 import pandas as pd
5
6 # Basic graph properties
7 print("Number of nodes:", G.number_of_nodes())
8 print("Number of edges:", G.number_of_edges())
9
10 # Read edge list from CSV
11 df_edges = pd.read_csv('graph_edges.csv')
12
13 # Create a graph from the edge list
14 G = nx.from_pandas_edgelist(df_edges, source='source', target='target')
15
16 # Print the graph
17 print(G.nodes(data=True))
18 print(G.edges(data=True))
19
20 # Draw the graph
21 nx.draw(G, with_labels=True)
22 plt.show()
  
```

Edge list and node attributes exported to CSV.

[('Sakshi', {'color': 'pink'}), ('Prim', {'color': 'red'}), ('Katniss', {'color': 'green'}), ('Gale', {'color': 'orange'}), ('Haymitch', {'color': 'purple'}), ('Lucy', {'color': 'indigo'}), ('Peeta', {'color': 'yellow'}), ('Caesar', {'color': 'violet'}), ('Snow', {'color': 'blue'}), ('Effie', {'color': 'grey'})]

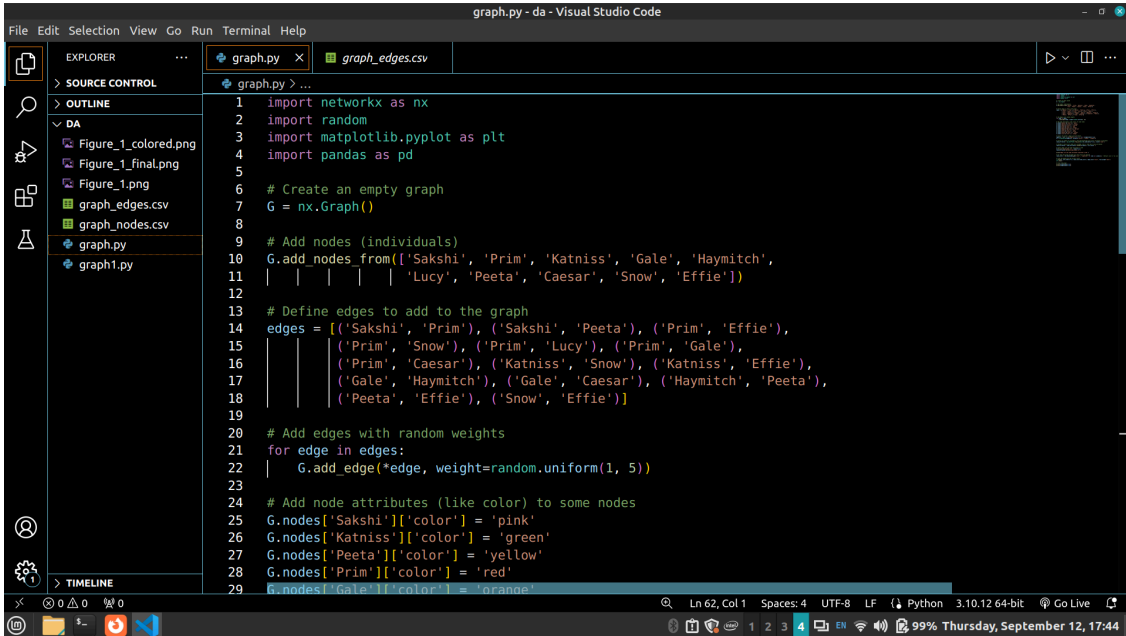
[('Sakshi', 'Prim', {'weight': 3.0226877535990413}), ('Sakshi', 'Peeta', {'weight': 4.4983396511560905}), ('Prim', 'Effie', {'weight': 4.150127964265766}), ('Prim', 'Snow', {'weight': 1.120907671661008}), ('Prim', 'Lucy', {'weight': 4.685303704904885}), ('Prim', 'Gale', {'weight': 3.456893683355261}), ('Prim', 'Caesar', {'weight': 4.936059464704805}), ('Katniss', 'Snow', {'weight': 4.928907738481952}), ('Katniss', 'Effie', {'weight': 3.190104643296904}), ('Gale', 'Haymitch', {'weight': 2.5244106970918856}), ('Gale', 'Caesar', {'weight': 1.828424847225023}), ('Haymitch', 'Peeta', {'weight': 4.707886871935422}), ('Peeta', 'Effie', {'weight': 3.474121032023563}), ('Snow', 'Effie', {'weight': 1.3736947184270134})]

Number of nodes: 10

Number of edges: 14

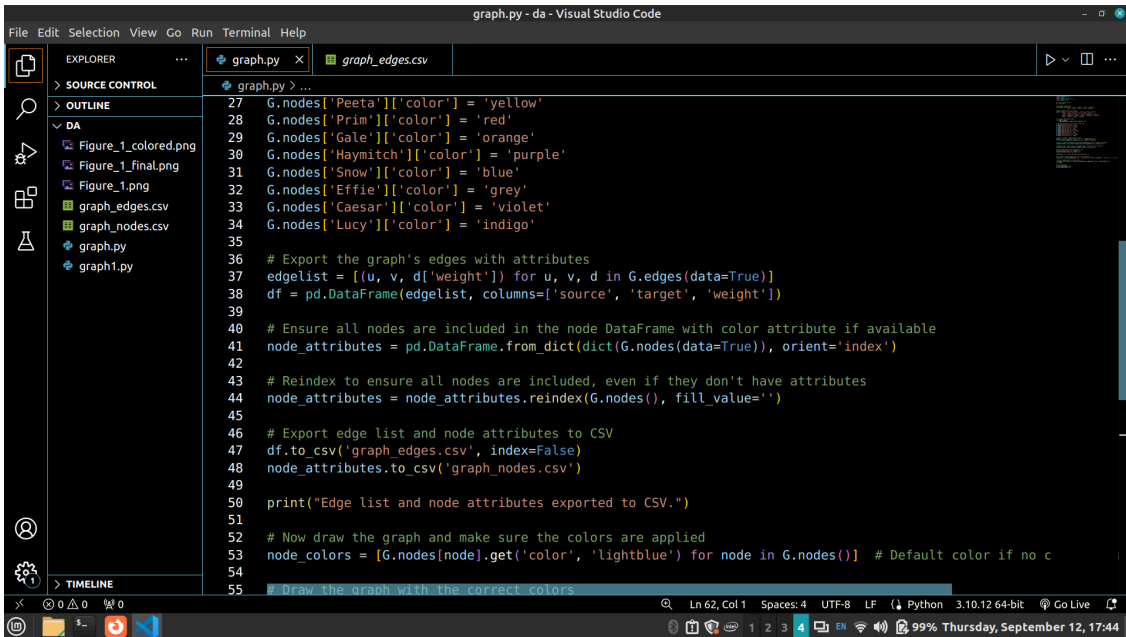
[('Sakshi', {}), ('Prim', {}), ('Peeta', {}), ('Effie', {}), ('Snow', {}), ('Lucy', {}), ('Gale', {}), ('Caesar', {}), ('Katniss', {}), ('Haymitch', {})]

[('Sakshi', 'Prim', {}), ('Sakshi', 'Peeta', {}), ('Prim', 'Effie', {}), ('Prim', 'Snow', {}), ('Prim', 'Lucy', {}), ('Prim', 'Gale', {}), ('Prim', 'Caesar', {}), ('Peeta', 'Haymitch', {}), ('Peeta', 'Effie', {}), ('Effie', 'Katniss', {}), ('Effie', 'Snow', {}), ('Snow', 'Katniss', {}), ('Gale', 'Haymitch', {}), ('Gale', 'Caesar', {})]



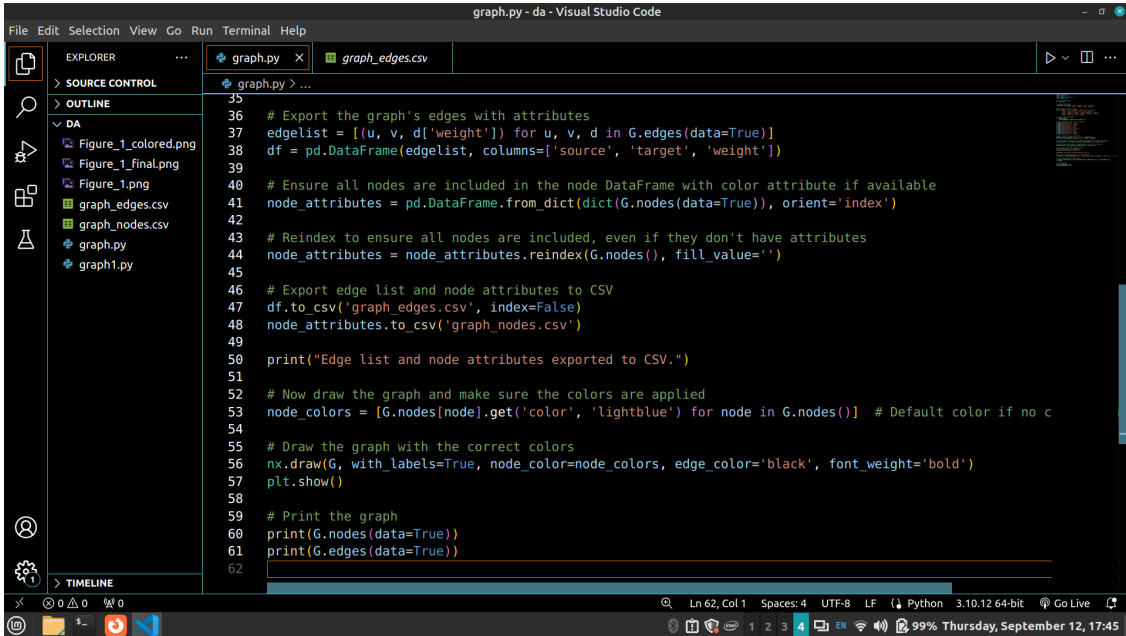
```

1 import networkx as nx
2 import random
3 import matplotlib.pyplot as plt
4 import pandas as pd
5
6 # Create an empty graph
7 G = nx.Graph()
8
9 # Add nodes (individuals)
10 G.add_nodes_from(['Sakshi', 'Prim', 'Katniss', 'Gale', 'Haymitch',
11 | | | | 'Lucy', 'Peeta', 'Caesar', 'Snow', 'Effie'])
12
13 # Define edges to add to the graph
14 edges = [('Sakshi', 'Prim'), ('Sakshi', 'Peeta'), ('Prim', 'Effie'),
15 | | | | ('Prim', 'Snow'), ('Prim', 'Lucy'), ('Prim', 'Gale'),
16 | | | | ('Prim', 'Caesar'), ('Katniss', 'Snow'), ('Katniss', 'Effie'),
17 | | | | ('Gale', 'Haymitch'), ('Gale', 'Caesar'), ('Haymitch', 'Peeta'),
18 | | | | ('Peeta', 'Effie'), ('Snow', 'Effie')]
19
20 # Add edges with random weights
21 for edge in edges:
22 | G.add_edge(*edge, weight=random.uniform(1, 5))
23
24 # Add node attributes (like color) to some nodes
25 G.nodes['Sakshi']['color'] = 'pink'
26 G.nodes['Katniss']['color'] = 'green'
27 G.nodes['Peeta']['color'] = 'yellow'
28 G.nodes['Prim']['color'] = 'red'
29 G.nodes['Gale']['color'] = 'orange'
  
```



```

27 G.nodes['Peeta']['color'] = 'yellow'
28 G.nodes['Prim']['color'] = 'red'
29 G.nodes['Gale']['color'] = 'orange'
30 G.nodes['Haymitch']['color'] = 'purple'
31 G.nodes['Snow']['color'] = 'blue'
32 G.nodes['Effie']['color'] = 'grey'
33 G.nodes['Caesar']['color'] = 'violet'
34 G.nodes['Lucy']['color'] = 'indigo'
35
36 # Export the graph's edges with attributes
37 edgelist = [(u, v, d['weight']) for u, v, d in G.edges(data=True)]
38 df = pd.DataFrame(edgelist, columns=['source', 'target', 'weight'])
39
40 # Ensure all nodes are included in the node DataFrame with color attribute if available
41 node_attributes = pd.DataFrame.from_dict(dict(G.nodes(data=True)), orient='index')
42
43 # Reindex to ensure all nodes are included, even if they don't have attributes
44 node_attributes = node_attributes.reindex(G.nodes(), fill_value='')
45
46 # Export edge list and node attributes to CSV
47 df.to_csv('graph_edges.csv', index=False)
48 node_attributes.to_csv('graph_nodes.csv')
49
50 print("Edge list and node attributes exported to CSV.")
51
52 # Now draw the graph and make sure the colors are applied
53 node_colors = [G.nodes[node].get('color', 'lightblue') for node in G.nodes()] # Default color if no c
54
55 # Draw the graph with the correct colors
  
```



```

35
36 # Export the graph's edges with attributes
37 edgelist = [(u, v, d['weight']) for u, v, d in G.edges(data=True)]
38 df = pd.DataFrame(edgelist, columns=['source', 'target', 'weight'])
39
40 # Ensure all nodes are included in the node DataFrame with color attribute if available
41 node_attributes = pd.DataFrame.from_dict(dict(G.nodes(data=True)), orient='index')
42
43 # Reindex to ensure all nodes are included, even if they don't have attributes
44 node_attributes = node_attributes.reindex(G.nodes(), fill_value='')
45
46 # Export edge list and node attributes to CSV
47 df.to_csv('graph_edges.csv', index=False)
48 node_attributes.to_csv('graph_nodes.csv')
49
50 print("Edge list and node attributes exported to CSV.")
51
52 # Now draw the graph and make sure the colors are applied
53 node_colors = [G.nodes[node].get('color', 'lightblue') for node in G.nodes()] # Default color if no c
54
55 # Draw the graph with the correct colors
56 nx.draw(G, with_labels=True, node_color=node_colors, edge_color='black', font_weight='bold')
57 plt.show()
58
59 # Print the graph
60 print(G.nodes(data=True))
61 print(G.edges(data=True))
62
  
```

[('Sakshi', {'color': 'pink'}), ('Prim', {'color': 'red'}), ('Katniss', {'color': 'green'}), ('Gale', {'color': 'orange'}), ('Haymitch', {'color': 'purple'}), ('Lucy', {'color': 'indigo'}), ('Peeta', {'color': 'yellow'}), ('Caesar', {'color': 'violet'}), ('Snow', {'color': 'blue'}), ('Effie', {'color': 'grey'})]

[('Sakshi', 'Prim', {'weight': 2.6962280723875787}), ('Sakshi', 'Peeta', {'weight': 1.9386672230656115}), ('Prim', 'Effie', {'weight': 1.2070108561861619}), ('Prim', 'Snow', {'weight': 1.5225606264590983}), ('Prim', 'Lucy', {'weight': 2.706470954529101}), ('Prim', 'Gale', {'weight': 1.0490699257294973}), ('Prim', 'Caesar', {'weight': 1.2659587830508712}), ('Katniss', 'Snow', {'weight': 2.134163240976767}), ('Katniss', 'Effie', {'weight': 1.1206689510597356}), ('Gale', 'Haymitch', {'weight': 3.2259333762638716}), ('Gale', 'Caesar', {'weight': 2.0816564913446065}), ('Haymitch', 'Peeta', {'weight': 4.710904755582943}), ('Peeta', 'Effie', {'weight': 3.509908889670192}), ('Snow', 'Effie', {'weight': 1.7202124262992804})]

Number of nodes: 10

Number of edges: 14

Degree Centrality: {'Sakshi': 2, 'Prim': 6, 'Katniss': 2, 'Gale': 3, 'Haymitch': 2, 'Lucy': 1, 'Peeta': 3, 'Caesar': 2, 'Snow': 3, 'Effie': 4}

Betweenness Centrality: {'Sakshi': 1.3333333333333333, 'Prim': 19.333333333333332, 'Katniss': 0.0, 'Gale': 3.833333333333333, 'Haymitch': 1.3333333333333333, 'Lucy': 0.0, 'Peeta': 4.333333333333333, 'Caesar': 0.0, 'Snow': 2.333333333333333, 'Effie': 7.499999999999999}

Normalized Betweenness Centrality: {'Sakshi': 0.037037037037037035, 'Prim': 0.537037037037037, 'Katniss': 0.0, 'Gale': 0.10648148148148147, 'Haymitch': 0.037037037037037035, 'Lucy': 0.0, 'Peeta': 0.12037037037037035, 'Caesar': 0.0, 'Snow': 0.0648148148148148, 'Effie': 0.2083333333333333}

Edge Betweenness Centrality: {'Sakshi', 'Prim': 7.5, ('Sakshi', 'Peeta'): 4.166666666666666, ('Prim', 'Effie'): 8.166666666666668, ('Prim', 'Snow'): 7.833333333333333, ('Prim', 'Lucy'): 9.0, ('Prim', 'Gale'): 8.5, ('Prim', 'Caesar'): 6.666666666666666, ('Katniss', 'Snow'): 3.333333333333333, ('Katniss', 'Effie'): 5.666666666666666, ('Gale', 'Haymitch'): 5.833333333333334, ('Gale', 'Caesar'): 2.333333333333333, ('Haymitch', 'Peeta'): 5.833333333333333, ('Peeta', 'Effie'): 7.666666666666667, ('Snow', 'Effie'): 2.5}

Normalized Edge Betweenness Centrality: {'Sakshi', 'Prim': 0.16666666666666669, ('Sakshi', 'Peeta'): 0.09259259259259259, ('Prim', 'Effie'): 0.18148148148148152, ('Prim', 'Snow'): 0.17407407407407408, ('Prim', 'Lucy'): 0.2, ('Prim', 'Gale'): 0.18888888888888888, ('Prim', 'Caesar'): 0.14814814814814814, ('Katniss', 'Snow'): 0.07407407407407407, ('Katniss', 'Effie'): 0.1259259259259259, ('Gale', 'Haymitch'): 0.12962962962962965, ('Gale', 'Caesar'): 0.05185185185185185, ('Haymitch', 'Peeta'): 0.12962962962962962, ('Peeta', 'Effie'): 0.1703703703703704, ('Snow', 'Effie'): 0.05555555555555556}

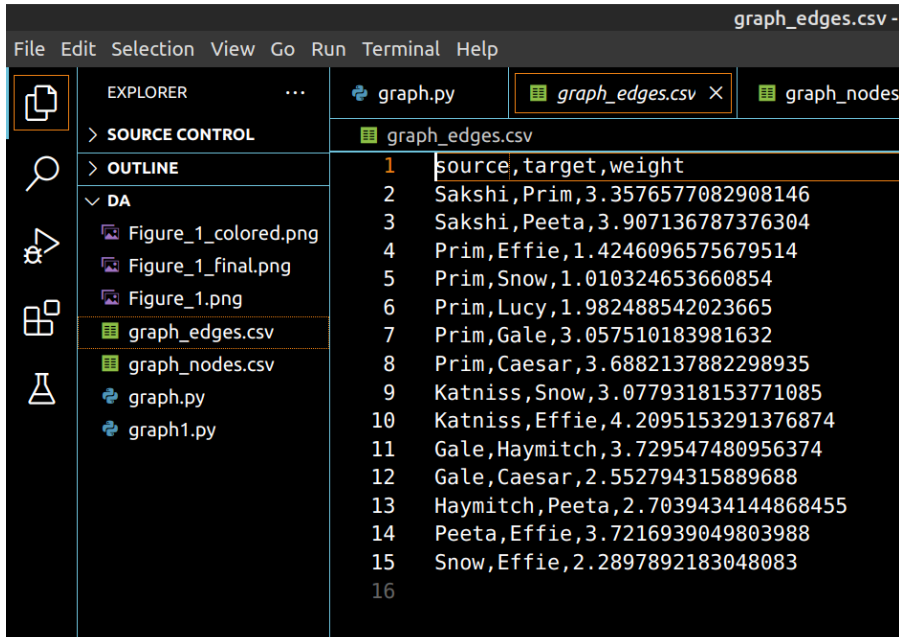
Communities after 1 step: ({'Prim', 'Effie', 'Gale', 'Haymitch', 'Katniss', 'Sakshi', 'Caesar', 'Peeta', 'Snow'}, {'Lucy'})

Communities after 2 steps: ({'Haymitch', 'Peeta', 'Sakshi'}, {'Prim', 'Effie', 'Gale', 'Katniss', 'Caesar', 'Snow'}, {'Lucy'})

Communities after 3 steps: ({'Haymitch', 'Peeta', 'Sakshi'}, {'Caesar', 'Gale', 'Prim'}, {'Effie', 'Katniss', 'Snow'}, {'Lucy'})

Communities after 4 steps: ({'Sakshi'}, {'Caesar', 'Gale', 'Prim'}, {'Effie', 'Katniss', 'Snow'}, {'Haymitch', 'Peeta'}, {'Lucy'})

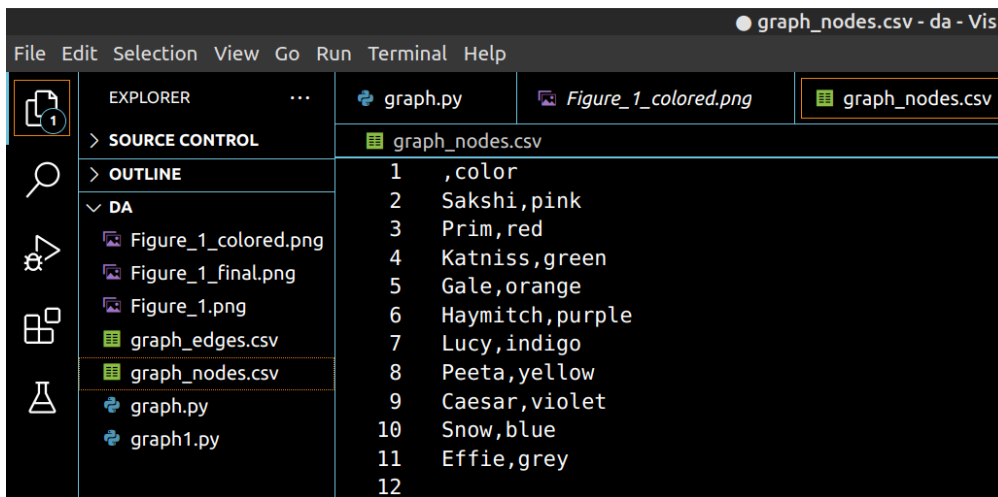
Communities after 5 steps: ({'Sakshi'}, {'Prim'}, {'Effie', 'Katniss', 'Snow'}, {'Caesar', 'Gale'}, {'Haymitch', 'Peeta'}, {'Lucy'})



The screenshot shows the VS Code editor with the file `graph_edges.csv` open. The Explorer sidebar on the left shows the project structure with files like `Figure_1_colored.png`, `Figure_1_final.png`, `Figure_1.png`, `graph_edges.csv`, `graph_nodes.csv`, `graph.py`, and `graph1.py`. The main editor area displays the content of `graph_edges.csv`:

```

1 source,target,weight
2 Sakshi,Prim,3.3576577082908146
3 Sakshi,Peeta,3.907136787376304
4 Prim,Effie,1.4246096575679514
5 Prim,Snow,1.010324653660854
6 Prim,Lucy,1.982488542023665
7 Prim,Gale,3.057510183981632
8 Prim,Caesar,3.6882137882298935
9 Katniss,Snow,3.0779318153771085
10 Katniss,Effie,4.2095153291376874
11 Gale,Haymitch,3.729547480956374
12 Gale,Caesar,2.552794315889688
13 Haymitch,Peeta,2.7039434144868455
14 Peeta,Effie,3.7216939049803988
15 Snow,Effie,2.2897892183048083
16
  
```



The screenshot shows the VS Code editor with the file `graph_nodes.csv` open. The Explorer sidebar on the left shows the project structure with files like `Figure_1_colored.png`, `Figure_1_final.png`, `Figure_1.png`, `graph_edges.csv`, `graph_nodes.csv`, `graph.py`, and `graph1.py`. The main editor area displays the content of `graph_nodes.csv`:

```

1 ,color
2 Sakshi,pink
3 Prim,red
4 Katniss,green
5 Gale,orange
6 Haymitch,purple
7 Lucy,indigo
8 Peeta,yellow
9 Caesar,violet
10 Snow,blue
11 Effie,grey
12
  
```

Date: 3/ 10/ 2024

Signature of faculty in-charge

Post Lab Descriptive Questions:

1. Analyze the centrality measures you calculated. Which nodes were identified as the most influential? What does this mean in the context of the social network?

Answer:

In this experiment, we calculated several centrality measures to identify influential nodes in the social network:

- **Degree Centrality:** The degree centrality values indicate the number of direct connections each node has. For example, **Prim** has the highest degree centrality with 6 connections, making it a central figure in the network. Other nodes, like **Gale** and **Effie**, also have significant connections (3 and 4, respectively).
- **Betweenness Centrality:** This measure reveals nodes that serve as bridges within the network. **Prim** again stands out with a high betweenness centrality score (19.33), indicating it plays a critical role in connecting disparate parts of the network. Other notable nodes include **Effie** and **Peeta**, which also show substantial betweenness.
- **Closeness Centrality:** Not explicitly calculated in your output, this measure can be useful to identify how quickly a node can access others in the network.

Contextual Meaning: In the context of a social network, nodes like **Prim** are likely influential in spreading information or coordinating activities, as they are well-connected and serve as conduits between groups.

2. Describe the communities identified using the Girvan-Newman algorithm. What are the characteristics of these communities? How do they relate to the social network's structure?

Answer:

The Girvan-Newman algorithm revealed several communities as edges with the highest betweenness centrality were removed:

- **First Split:** The first division identified a large community containing most nodes, including **Prim**, **Gale**, **Effie**, and others, alongside a smaller community with **Lucy**.
- **Subsequent Splits:** As the algorithm progressed, it became evident that smaller, cohesive groups formed. For instance, after several splits, communities were identified where **Haymitch** and **Peeta** clustered together, while **Effie**, **Katniss**, and **Snow** formed another group.

Characteristics of Communities:

- **Cohesion:** Communities are characterized by tight interconnections, suggesting shared interests or mutual influences.
- **Isolated Nodes:** **Lucy** often stands alone, indicating weaker ties to the rest of the network.

Relation to Network Structure: The structure reflects social dynamics where certain groups may have shared interests or activities, while others may be more isolated. The community structure can inform the flow of information and potential collaborations.

3. Discuss the implications of identifying influential nodes in the network. How can this information be used?

Answer:

Identifying influential nodes has several practical implications:

- **Targeted Communication:** Knowledge of influential individuals can aid in designing communication strategies. For instance, messages or campaigns can be directed at nodes like **Prim** to maximize outreach.

- **Resource Allocation:** In organizational settings, focusing on high-betweenness nodes for leadership roles can help bridge gaps between departments or teams.
- **Intervention Strategies:** In public health or marketing, leveraging influential individuals can help in disseminating information quickly and effectively.
- **Network Resilience:** Understanding which nodes are pivotal for connectivity allows for strategic planning in case of disruptions. If an influential node is removed, alternative connections can be reinforced to maintain network functionality.